Salida
- depreciación anual por año
- depreciación acumulada en cada año
- valor del automóvil en cada año

Proceso
- depreciación acumulada
- cálculo de la depreciación acumulada cada año
- cálculo del valor del automóvil en cada año

La tabla siguiente muestra la salida solicitada

Año	Depreciación	Depreciación acumulada	Valor anual
1 (2006)	3.000	3.000	17.000
2 (2007)	3.000	6.000	14.000
3 (2008)	3.000	9.000	11.000
4 (2009)	3.000	12.000	8.000
5 (2010)	3.000	15.000	5.000
6 (2011)	3.000	18.000	2.000

2.1.2. Diseño del algoritmo

En la etapa de análisis del proceso de programación se determina *qué* hace el programa. En la etapa de diseño se determina *cómo* hace el programa la tarea solicitada. Los métodos más eficaces para el proceso de diseño se basan en el conocido *divide y vencerás*. Es decir, la resolución de un problema complejo se realiza dividiendo el problema en subproblemas y a continuación dividiendo estos subproblemas en otros de nivel más bajo, hasta que pueda ser *implementada* una solución en la computadora. Este método se conoce técnicamente como **diseño descendente** (*top-down*) o **modular**. El proceso de romper el problema en cada etapa y expresar cada paso en forma más detallada se denomina *refinamiento sucesivo*.

Cada subprograma es resuelto mediante un **módulo** (*subprograma*) que tiene un solo punto de entrada y un solo punto de salida.

Cualquier programa bien diseñado consta de un *programa principal* (el módulo de nivel más alto) que llama a subprogramas (módulos de nivel más bajo) que a su vez pueden llamar a otros subprogramas. Los programas estructurados de esta forma se dice que tienen un *diseño modular* y el método de romper el programa en módulos más pequeños se llama *programación modular*. Los módulos pueden ser planeados, codificados, comprobados y depurados independientemente (incluso por diferentes programadores) y a continuación combinarlos entre sí. El proceso implica la ejecución de los siguientes pasos hasta que el programa se termina:

1. Programar un módulo.
2. Comprobar el módulo.
3. Si es necesario, depurar el módulo.
4. Combinar el módulo con los módulos anteriores.

El proceso que convierte los resultados del análisis del problema en un diseño modular con refinamientos sucesivos que permitan una posterior traducción a un lenguaje se denomina **diseño del algoritmo**.

El diseño del algoritmo es independiente del lenguaje de programación en el que se vaya a codificar posteriormente.

2.1.3. Herramientas de programación

Las dos herramientas más utilizadas comúnmente para diseñar algoritmos son: *diagramas de flujo* y *pseudocódigos*.

Un **diagrama de flujo** (*flowchart*) es una representación gráfica de un algoritmo. Los símbolos utilizados han sido normalizados por el Instituto Norteamericano de Normalización (**ANSI**), y los más frecuentemente empleados se muestran en la Figura 2.2, junto con una plantilla utilizada para el dibujo de los diagramas de flujo (Figura 2.3). En la Figura 2.4 se representa el diagrama de flujo que resuelve el Problema 2.1.

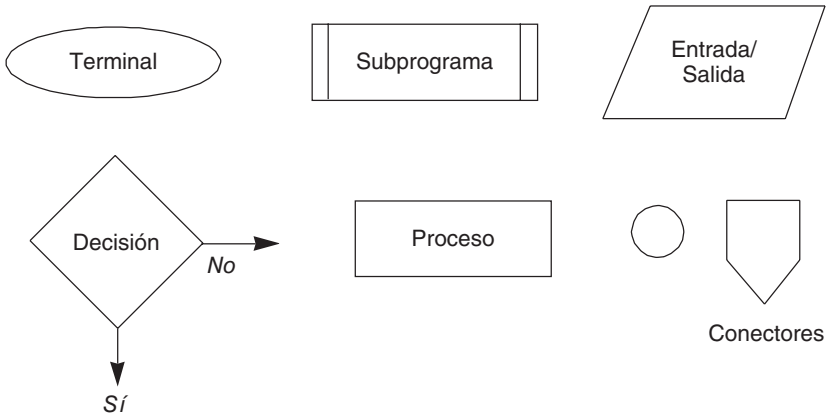

Figura 2.2. Símbolos más utilizados en los diagramas de flujo.

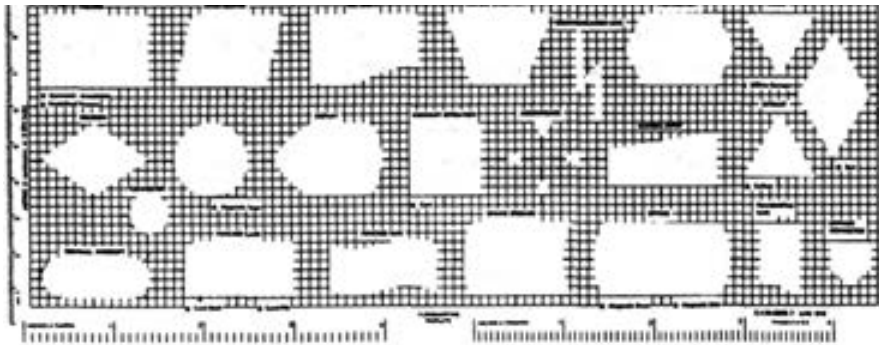

Figura 2.3. Plantilla para dibujo de diagramas de flujo.

El **pseudocódigo** es una herramienta de programación en la que las instrucciones se escriben en palabras similares al inglés o español, que facilitan tanto la escritura como la lectura de programas. En esencia, el pseudocódigo se puede definir como *un lenguaje de especificaciones de algoritmos*.

Aunque no existen reglas para escritura del pseudocódigo en español, se ha recogido una notación estándar que se utilizará en el libro y que ya es muy empleada en los libros de programación en español[3]. Las palabras reservadas básicas se representarán en letras negritas minúsculas. Estas palabras son traducción libre de palabras reservadas de lenguajes como C, Pascal, etc. Más adelante se indicarán los pseudocódigos fundamentales para utilizar en esta obra.

El pseudocódigo que resuelve el Problema 2.1 es:

```
Previsiones de depreciacion
Introducir coste
          vida util
          valor final de rescate (recuperacion)
imprimir cabeceras
Establecer el valor inicial del año
Calcular depreciacion
```

[3] Para mayor ampliación sobre el *pseudocódigo,* puede consultar, entre otras, algunas de estas obras: *Fundamentos de programación,* Luis Joyanes, 2.ª edición, 1997; *Metodología de la programación,* Luis Joyanes, 1986; *Problemas de Metodología de la programación,* Luis Joyanes, 1991 (todas ellas publicadas en McGraw-Hill, Madrid), así como *Introducción a la programación,* de Clavel y Biondi. Barcelona: Masson, 1987, o bien *Introducción a la programación y a las estructuras de datos,* de Braunstein y Groia. Buenos Aires: Editorial Eudeba, 1986. Para una formación práctica puede consultar: *Fundamentos de programación: Libro de problemas* de Luis Joyanes, Luis Rodríguez y Matilde Fernández, en McGraw-Hill (Madrid, 1998).

```
mientras valor año =< vida util hacer
      calcular depreciacion acumulada
      calcular valor actual
      imprimir una linea en la tabla
      incrementar el valor del año
fin de mientras
```

EJEMPLO 2.1

Calcular la paga neta de un trabajador conociendo el número de horas trabajadas, la tarifa horaria y la tasa de impuestos.

```
Algoritmo
   1. Leer Horas, Tarifa, Tasa
   2. Calcular PagaBruta = Horas * Tarifa
   3. Calcular Impuestos = PagaBruta * Tasa
   4. Calcular PagaNeta = PagaBruta - Impuestos
   5. Visualizar PagaBruta, Impuestos, PagaNeta
```

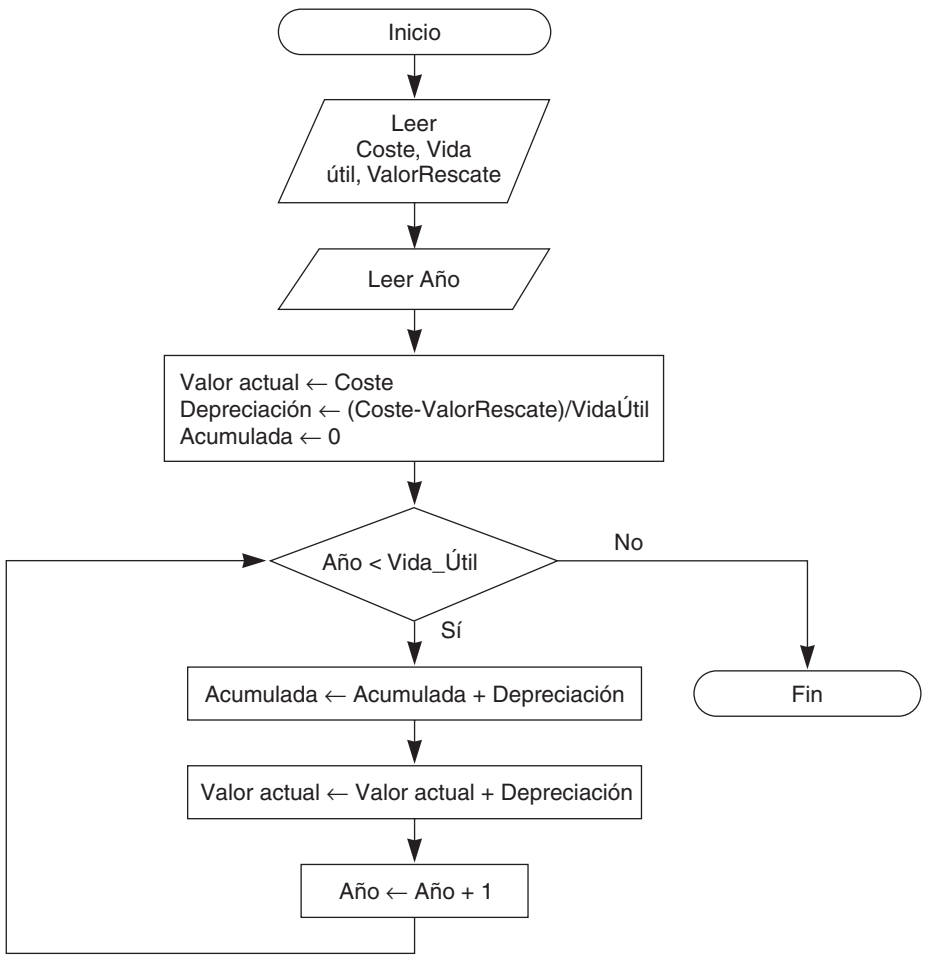

Figura 2.4. Diagrama de flujo (Problema 2.1).

EJEMPLO 2.2

Calcular el valor de la suma 1+2+3+...+100.

algoritmo
Se utiliza una variable `Contador` como un contador que genere los sucesivos números enteros, y `Suma` para almacenar las sumas parciales 1, 1+2, 1+2+3…

```
1. Establecer Contador a 1
2. Establecer Suma a 0
3. mientras Contador <= 100 hacer
     Sumar Contador a Suma
     Incrementar Contador en 1
   fin_mientras
4. Visualizar Suma
```

2.1.4. Codificación de un programa

La *codificación* es la escritura en un lenguaje de programación de la representación del algoritmo desarrollada en las etapas precedentes. Dado que el diseño de un algoritmo es independiente del lenguaje de programación utilizado para su implementación, el código puede ser escrito con igual facilidad en un lenguaje o en otro.

Para realizar la conversión del algoritmo en programa se deben sustituir las palabras reservadas en español por sus homónimos en inglés, y las operaciones/instrucciones indicadas en lenguaje natural por el lenguaje de programación correspondiente.

```
{Este programa obtiene una tabla de depreciaciones
 acumuladas y valores reales de cada año de un
 determinado producto}

algoritmo primero
  Real: Coste, Depreciacion,
        Valor_Recuperacion
        Valor_Actual,
        Acumulado
        Valor_Anual;
  entero: Año, Vida_Util;
  inicio
        escribir('introduzca coste, valor recuperación y vida útil')
        leer(Coste, Valor_Recuperacion, Vida_Util)
        escribir('Introduzca año actual')
        leer(Año)
        Valor_Actual ← Coste;
        Depreciacion ← (Coste-Valor_Recuperacion)/Vida_Util
        Acumulado ← 0
        escribir('Año Depreciación Dep. Acumulada')
        mientras (Año < Vida_Util)
           Acumulado ← Acumulado + Depreciacion
           Valor_Actual ← Valor_Actual - Depreciacion
           escribir('Año, Depreciacion, Acumulado')
           Año ← Año + 1;
        fin mientras
  fin
```